Redux

Table of Contents

Redux Toolkit TypeScript

Step 1: Create a Slice with TypeScript

TS

```ts
// features/counter/counterSlice.ts
import { createSlice, PayloadAction } from '@reduxjs/toolkit';

interface CounterState {
  value: number;
  status: 'idle' | 'loading';
}

const initialState: CounterState = {
  value: 0,
  status: 'idle',
};

const counterSlice = createSlice({
  name: 'counter',
  initialState,
  reducers: {
    increment: (state) => {
      state.value += 1;
    },
    decrement: (state) => {
      state.value -= 1;
    },
    incrementByAmount: (state, action: PayloadAction<number>) => {
      state.value += action.payload;
    },
  },
});

export const { increment, decrement, incrementByAmount } = counterSlice.a
export default counterSlice.reducer;
```

Step 2: Configure Store and Export Types

TS

```ts
// store/store.ts
import { configureStore } from '@reduxjs/toolkit';
import counterReducer from '../features/counter/counterSlice';

export const store = configureStore({
  reducer: {
    counter: counterReducer,
  },
});

export type RootState = ReturnType<typeof store.getState>;
export type AppDispatch = typeof store.dispatch;
export type AppStore = typeof store;
```

Step 3: Create Typed Hooks

TS

```ts
// store/hooks.ts
import { TypedUseSelectorHook, useDispatch, useSelector } from 'react-rec
import { AppDispatch, RootState } from './store';

export const useAppDispatch = () => useDispatch<AppDispatch>();

export const useAppSelector: TypedUseSelectorHook<RootState> = useSelecto
```

Step 4: Provide Store

TSX

```tsx
// main.tsx
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import { Provider } from 'react-redux';
import { store } from './store/store';
import App from './App';

createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <Provider store={store}>
      <App />
    </Provider>
  </StrictMode>
);
```

Step 5: Use in Components

TSX

```tsx
// features/counter/Counter.tsx
import { useAppDispatch, useAppSelector } from '../../store/hooks';
import { increment, decrement, incrementByAmount } from './counterSlice';

const Counter = () => {
  const { value, status } = useAppSelector((state) => state.counter);
  const dispatch = useAppDispatch();

  return (
    <div>
      <p>Value: {value}</p>
      <p>Status: {status}</p>
      <button onClick={() => dispatch(increment())}>+</button>
      <button onClick={() => dispatch(decrement())}>-</button>
      <button onClick={() => dispatch(incrementByAmount(5))}>+5</button>
    </div>
  );
};

export default Counter;
```

Step 6: createAsyncThunk with TypeScript

TS

```ts
// features/items/itemsSlice.ts
import { createSlice, createAsyncThunk, PayloadAction } from '@reduxjs/to
import type { RootState } from '../../store/store';

interface Item {
  id: number;
  name: string;
}

interface ItemsState {
  items: Item[];
  status: 'idle' | 'loading' | 'succeeded' | 'failed';
  error: string | null;
}

const initialState: ItemsState = {
  items: [],
  status: 'idle',
  error: null,
};

export const fetchItems = createAsyncThunk<Item[], void, { state: RootSta
  'items/fetch',
  async () => {
    const response = await fetch('/api/items');
    return response.json();
  }
);

const itemsSlice = createSlice({
  name: 'items',
  initialState,
  reducers: {},
  extraReducers: (builder) => {
    builder
      .addCase(fetchItems.pending, (state) => {
        state.status = 'loading';
      })
      .addCase(fetchItems.fulfilled, (state, action: PayloadAction<Item[]
        state.status = 'succeeded';
        state.items = action.payload;
      })
      .addCase(fetchItems.rejected, (state, action) => {
        state.status = 'failed';
        state.error = action.error.message || 'Something went wrong';
      });
  },
```

```
});

export default itemsSlice.reducer;
```

Step 7: Use Async in Component

TSX

```tsx
// features/items/ItemsList.tsx
import { useEffect } from 'react';
import { useAppDispatch, useAppSelector } from '../../store/hooks';
import { fetchItems } from './itemsSlice';

const ItemsList = () => {
  const dispatch = useAppDispatch();
  const { items, status, error } = useAppSelector((state) => state.items)

  useEffect(() => {
    dispatch(fetchItems());
  }, [dispatch]);

  if (status === 'loading') return <p>Loading...</p>;
  if (status === 'failed') return <p>Error: {error}</p>;

  return (
    <ul>
      {items.map((item) => (
        <li key={item.id}>{item.name}</li>
      ))}
    </ul>
  );
};

export default ItemsList;
```

Redux Toolkit (RTK)

Redux Toolkit is the recommended way to write Redux logic. It includes utilities like `createSlice`, `configureStore`, `createAsyncThunk`, and RTK Query.

For a side-by-side comparison with traditional Redux, see Redux vs RTK.

For TypeScript setup, see Redux Toolkit TS.

Steps for Redux Toolkit

Step 1: Install Redux Toolkit and React-Redux

BASH

```bash
npm install @reduxjs/toolkit react-redux
```

Step 2: Create the Redux Store Using configureStore

The store is the central place where your application's state is stored. Redux Toolkit simplifies the store setup using the `configureStore` function.

JS

```js
// store.js
import { configureStore } from '@reduxjs/toolkit';
import counterReducer from './features/counter/counterSlice';

const store = configureStore({
    reducer: {
        counter: counterReducer,
    },
});

export default store;
```

Step 3: Create Slices with createSlice

Slices are a way to manage state and actions together in Redux Toolkit. `createSlice` automatically generates action creators and reducers.

JS

```
// counterSlice.js
import { createSlice } from '@reduxjs/toolkit';

const initialState = { value: 0 };

const counterSlice = createSlice({
    name: 'counter',
    initialState,
    reducers: {
        increment: (state) => {
            state.value += 1; // Direct mutation is safe with Immer
        },
        decrement: (state) => {
            state.value -= 1;
        },
        incrementByAmount: (state, action) => {
            state.value += action.payload;
        },
    },
});

export const { increment, decrement, incrementByAmount } = counterSlice.a
export default counterSlice.reducer;
```

Step 4: Provide the Redux Store to Your App

Use the `<Provider>` component from `react-redux` to pass the Redux store down the component tree. This allows all components to access the store.

JSX

```
// index.js
import React from 'react';
import ReactDOM from 'react-dom';
import { Provider } from 'react-redux';
import store from './store';

ReactDOM.render(
    <Provider store={store}>
        <App />
    </Provider>,
    document.getElementById('root')
);
```

Step 5: Access Redux Store in Components

Use `useSelector` to access state:

JSX

```jsx
import { useSelector } from 'react-redux';

const Counter = () => {
    const value = useSelector((state) => state.counter.value);

    return <div>Counter Value: {value}</div>;
};
```

Use `useDispatch` to dispatch actions:

JSX

```jsx
import { useDispatch } from 'react-redux';
import { increment, decrement } from './features/counter/counterSlice';

const CounterControls = () => {
    const dispatch = useDispatch();

    return (
        <div>
            <button onClick={() => dispatch(increment())}>Increment</butt
            <button onClick={() => dispatch(decrement())}>Decrement</butt
        </div>
    );
};
```

Step 6: Handle Asynchronous Actions (Optional)

For async operations like API calls, use `createAsyncThunk`.

JS

```js
// fetchItems.js
import { createAsyncThunk } from '@reduxjs/toolkit';

export const fetchItems = createAsyncThunk('items/fetch', async () => {
    const response = await fetch('/api/items');
    return response.json();
});
```

Handle async actions in a slice:

JS

```
import { createSlice } from '@reduxjs/toolkit';
import { fetchItems } from './fetchItems';

const itemsSlice = createSlice({
    name: 'items',
    initialState: { items: [], status: 'idle' },
    reducers: {},
    extraReducers: (builder) => {
        builder
            .addCase(fetchItems.pending, (state) => {
                state.status = 'loading';
            })
            .addCase(fetchItems.fulfilled, (state, action) => {
                state.status = 'succeeded';
                state.items = action.payload;
            })
            .addCase(fetchItems.rejected, (state) => {
                state.status = 'failed';
            });
    },
});

export default itemsSlice.reducer;
```

Step 7: Dispatch Asynchronous Actions

JSX

```
import { useDispatch } from 'react-redux';
import { fetchItems } from './features/items/fetchItems';

const ItemsList = () => {
    const dispatch = useDispatch();

    const loadItems = () => {
        dispatch(fetchItems());
    };

    return <button onClick={loadItems}>Load Items</button>;
};
```

Rules for Reducers

- Only modify state
- No side effects (no API calls, console.log, localStorage, async logic)

- Must be pure functions
- Must always return a new state

Redux Traditional Steps (JavaScript)

Step 1: Install Redux and React-Redux

BASH

```bash
npm install redux react-redux
```

Step 2: Create the Redux Store

JS

```js
// store.js
import { createStore } from 'redux';
import rootReducer from './reducers';

const store = createStore(rootReducer);

export default store;
```

Step 3: Create Reducers and Actions

Reducers

JS

```js
// reducers/itemsReducer.js
const initialState = { items: [] };

const itemsReducer = (state = initialState, action) => {
    switch (action.type) {
        case 'ADD_ITEM':
            return {
                ...state,
                items: [...state.items, action.payload],
            };
        default:
            return state;
    }
};

export default itemsReducer;
```

Actions

JS

```
// actions/itemsActions.js
export const addItem = (item) => ({
    type: 'ADD_ITEM',
    payload: item,
});
```

Step 4: Combine Reducers (if needed)

JS

```
// reducers/index.js
import { combineReducers } from 'redux';
import itemsReducer from './itemsReducer';

const rootReducer = combineReducers({
    items: itemsReducer,
});

export default rootReducer;
```

Step 5: Provide the Redux Store to Your App

JSX

```
// index.js
import React from 'react';
import ReactDOM from 'react-dom';
import { Provider } from 'react-redux';
import store from './store';
import App from './App';

ReactDOM.render(
    <Provider store={store}>
        <App />
    </Provider>,
    document.getElementById('root')
);
```

Step 6: Connect React Components to Redux

JSX

```jsx
// components/ItemsList.js
import { useSelector } from 'react-redux';

const ItemsList = () => {
    const items = useSelector((state) => state.items.items);

    return (
        <div>
            {items.map((item) => (
                <div key={item.id}>{item.name}</div>
            ))}
        </div>
    );
};

export default ItemsList;
```

JSX

```jsx
// components/AddItemForm.js
import { useDispatch } from 'react-redux';
import { addItem } from '../actions/itemsActions';

const AddItemForm = () => {
    const dispatch = useDispatch();

    const handleAddItem = (item) => {
        dispatch(addItem(item));
    };

    return (
        <button onClick={() => handleAddItem({ id: 1, name: 'Item 1' })}>
            Add Item
        </button>
    );
};

export default AddItemForm;
```

Step 7: Handle Asynchronous Actions (optional)

For async operations like API calls, use Redux Thunk.

BASH

```bash
npm install redux-thunk
```

JS

```
// store.js
import { createStore, applyMiddleware } from 'redux';
import thunk from 'redux-thunk';
import rootReducer from './reducers';

const store = createStore(rootReducer, applyMiddleware(thunk));

export default store;
```

JS

```
// actions/itemsActions.js
export const fetchItems = () => {
    return async (dispatch) => {
        dispatch({ type: 'ITEMS_LOADING' });

        try {
            const response = await fetch('/api/items');
            const data = await response.json();
            dispatch({ type: 'ITEMS_SUCCESS', payload: data });
        } catch (error) {
            dispatch({ type: 'ITEMS_ERROR', payload: error.message });
        }
    };
};
```

JS

```
// reducers/itemsReducer.js
const initialState = { items: [], loading: false, error: null };

const itemsReducer = (state = initialState, action) => {
    switch (action.type) {
        case 'ITEMS_LOADING':
            return { ...state, loading: true, error: null };
        case 'ITEMS_SUCCESS':
            return { ...state, loading: false, items: action.payload };
        case 'ITEMS_ERROR':
            return { ...state, loading: false, error: action.payload };
        default:
            return state;
    }
};

export default itemsReducer;
```

JSX

```jsx
// components/ItemsList.js
import { useEffect } from 'react';
import { useDispatch, useSelector } from 'react-redux';
import { fetchItems } from '../actions/itemsActions';

const ItemsList = () => {
    const dispatch = useDispatch();
    const { items, loading, error } = useSelector((state) => state.items)

    useEffect(() => {
        dispatch(fetchItems());
    }, [dispatch]);

    if (loading) return <p>Loading...</p>;
    if (error) return <p>Error: {error}</p>;

    return (
        <ul>
            {items.map((item) => (
                <li key={item.id}>{item.name}</li>
            ))}
        </ul>
    );
};

export default ItemsList;
```

Redux Traditional Steps (TypeScript)

Step 1: Install

BASH

```bash
npm install redux react-redux
npm install -D @types/react-redux
```

Step 2: Create Types for Actions

TS

```ts
// types/actions.ts
export const ADD_ITEM = 'ADD_ITEM';
export const ITEMS_LOADING = 'ITEMS_LOADING';
export const ITEMS_SUCCESS = 'ITEMS_SUCCESS';
export const ITEMS_ERROR = 'ITEMS_ERROR';

interface AddItemAction {
  type: typeof ADD_ITEM;
  payload: Item;
}

interface ItemsLoadingAction {
  type: typeof ITEMS_LOADING;
}

interface ItemsSuccessAction {
  type: typeof ITEMS_SUCCESS;
  payload: Item[];
}

interface ItemsErrorAction {
  type: typeof ITEMS_ERROR;
  payload: string;
}

export type ItemsActionTypes =
  | AddItemAction
  | ItemsLoadingAction
  | ItemsSuccessAction
  | ItemsErrorAction;
```

TS

```ts
// types/item.ts
export interface Item {
  id: number;
  name: string;
}
```

Step 3: Create Reducers and Actions

Actions

TS

```ts
// actions/itemsActions.ts
import {
  ADD_ITEM,
  ITEMS_LOADING,
  ITEMS_SUCCESS,
  ITEMS_ERROR,
  type ItemsActionTypes,
} from '../types/actions';
import type { Item } from '../types/item';

export const addItem = (item: Item): ItemsActionTypes => ({
  type: ADD_ITEM,
  payload: item,
});
```

Reducer

TS

```ts
// reducers/itemsReducer.ts
import {
  ADD_ITEM,
  ITEMS_LOADING,
  ITEMS_SUCCESS,
  ITEMS_ERROR,
  type ItemsActionTypes,
} from '../types/actions';
import type { Item } from '../types/item';

interface ItemsState {
  items: Item[];
  loading: boolean;
  error: string | null;
}

const initialState: ItemsState = {
  items: [],
  loading: false,
  error: null,
};

const itemsReducer = (
  state = initialState,
  action: ItemsActionTypes
): ItemsState => {
  switch (action.type) {
    case ADD_ITEM:
      return {
        ...state,
        items: [...state.items, action.payload],
      };
    case ITEMS_LOADING:
      return { ...state, loading: true, error: null };
    case ITEMS_SUCCESS:
      return { ...state, loading: false, items: action.payload };
    case ITEMS_ERROR:
      return { ...state, loading: false, error: action.payload };
    default:
      return state;
  }
};

export default itemsReducer;
```

Step 4: Combine Reducers (if needed)

TS

```ts
// reducers/index.ts
import { combineReducers } from 'redux';
import itemsReducer from './itemsReducer';

const rootReducer = combineReducers({
  items: itemsReducer,
});

export default rootReducer;

export type RootState = ReturnType<typeof rootReducer>;
```

Step 5: Create the Store

TS

```ts
// store.ts
import { createStore } from 'redux';
import rootReducer from './reducers';

const store = createStore(rootReducer);

export default store;

export type AppDispatch = typeof store.dispatch;
```

Step 6: Provide the Store

TSX

```tsx
// index.tsx
import React from 'react';
import ReactDOM from 'react-dom';
import { Provider } from 'react-redux';
import store from './store';
import App from './App';

ReactDOM.render(
  <Provider store={store}>
    <App />
  </Provider>,
  document.getElementById('root')
);
```

Step 7: Connect Components

TSX

```tsx
// components/ItemsList.tsx
import { useSelector } from 'react-redux';
import type { RootState } from '../reducers';

const ItemsList = () => {
  const items = useSelector((state: RootState) => state.items.items);

  return (
    <div>
      {items.map((item) => (
        <div key={item.id}>{item.name}</div>
      ))}
    </div>
  );
};

export default ItemsList;
```

TSX

```tsx
// components/AddItemForm.tsx
import { useDispatch } from 'react-redux';
import { addItem } from '../actions/itemsActions';

const AddItemForm = () => {
  const dispatch = useDispatch();

  const handleAddItem = () => {
    dispatch(addItem({ id: 1, name: 'Item 1' }));
  };

  return <button onClick={handleAddItem}>Add Item</button>;
};

export default AddItemForm;
```

Step 8: Async with Redux Thunk

BASH

```bash
npm install redux-thunk
```

TS

```ts
// store.ts
import { createStore, applyMiddleware } from 'redux';
import thunk from 'redux-thunk';
import rootReducer from './reducers';

const store = createStore(rootReducer, applyMiddleware(thunk));

export default store;
export type AppDispatch = typeof store.dispatch;
```

TS

```ts
// actions/itemsActions.ts
import { ThunkAction } from 'redux-thunk';
import type { RootState } from '../reducers';
import {
  ITEMS_LOADING,
  ITEMS_SUCCESS,
  ITEMS_ERROR,
  type ItemsActionTypes,
} from '../types/actions';
import type { Item } from '../types/item';

export const fetchItems = (): ThunkAction<
  void,
  RootState,
  unknown,
  ItemsActionTypes
> => {
  return async (dispatch) => {
    dispatch({ type: ITEMS_LOADING });

    try {
      const response = await fetch('/api/items');
      const data: Item[] = await response.json();
      dispatch({ type: ITEMS_SUCCESS, payload: data });
    } catch (error) {
      dispatch({
        type: ITEMS_ERROR,
        payload: (error as Error).message,
      });
    }
  };
};
```

TSX

```tsx
// components/ItemsList.tsx
import { useEffect } from 'react';
import { useDispatch, useSelector } from 'react-redux';
import type { RootState } from '../reducers';
import { fetchItems } from '../actions/itemsActions';

const ItemsList = () => {
  const dispatch = useDispatch();
  const { items, loading, error } = useSelector(
    (state: RootState) => state.items
  );

  useEffect(() => {
    dispatch(fetchItems());
  }, [dispatch]);

  if (loading) return <p>Loading...</p>;
  if (error) return <p>Error: {error}</p>;

  return (
    <ul>
      {items.map((item) => (
        <li key={item.id}>{item.name}</li>
      ))}
    </ul>
  );
};

export default ItemsList;
```

Comparación: Redux vs Redux Toolkit (RTK)

Redux

JS

```
// actions.js
export const increment = () => ({ type: 'INCREMENT' });
export const decrement = () => ({ type: 'DECREMENT' });

// reducer.js
const initialState = { count: 0 };

function counterReducer(state = initialState, action) {
    switch (action.type) {
        case 'INCREMENT':
            return { ...state, count: state.count + 1 };
        case 'DECREMENT':
            return { ...state, count: state.count - 1 };
        default:
            return state;
    }
}
export default counterReducer;

// store.js
import { createStore } from 'redux';
import counterReducer from './reducer';

const store = createStore(counterReducer);

export default store;
```

Paso 2: Usar las acciones en un componente

JSX

```jsx
import { useDispatch, useSelector } from 'react-redux';
import { increment, decrement } from './actions';

const Counter = () => {
    const count = useSelector(state => state.count);
    const dispatch = useDispatch();

    return (
        <div>
            <p>Count: {count} </p>
            <button onClick={() => dispatch(increment())}>Increment</butt
            <button onClick={() => dispatch(decrement())}>Decrement</butt
        </div>
    );
};

export default Counter;
```

Redux Toolkit

JS

```js
// counterSlice.js
import { createSlice } from '@reduxjs/toolkit';

const counterSlice = createSlice({
    name: 'counter',
    initialState: { count: 0 },
    reducers: {
        increment: (state) => {
            state.count += 1;
        },
        decrement: (state) => {
            state.count -= 1;
        },
    },
});

export const { increment, decrement } = counterSlice.actions;
export default counterSlice.reducer;
```

Paso 2: Configuración del store

Con Redux Toolkit, configuramos el store de manera muy sencilla utilizando `configureStore`.

JS

```
// store.js
import { configureStore } from '@reduxjs/toolkit';
import counterReducer from './counterSlice';

const store = configureStore({
    reducer: {
        counter: counterReducer,
    },
});
export default store;
```

Paso 3: Usar las acciones en un componente

JSX

```
import React from 'react';
import { useDispatch, useSelector } from 'react-redux';
import { increment, decrement } from './counterSlice';

const Counter = () => {
    const count = useSelector(state => state.counter.count);
    const dispatch = useDispatch();

    return (
        <div>
            <p>Count: {count}</p>
            <button onClick={() => dispatch(increment())}>Increment</butt
            <button onClick={() => dispatch(decrement())}>Decrement</butt
        </div>
    );
};

export default Counter;
```

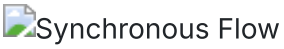
Redux

Core Concepts

- **Immutability**: The state in Redux is immutable, meaning it cannot be changed directly. Instead, a new state is created every time a change occurs. This helps avoid side effects and makes debugging easier.
- **Actions**: Actions are plain JavaScript objects that describe what happened in the application. Every action has at least a `type` property, which identifies the action, and optionally a `payload`, which holds the data to update the state. Actions are the only way to send data to the store. Actions are dispatched using `dispatch()`.
- **Reducers**: Reducers are pure functions that take the previous state and an action, and return a new state. They are responsible for specifying how the state changes in response to an action.
- **Unidirectional Data Flow (One-way data flow)**: Redux follows a unidirectional data flow. Actions are dispatched to the store, reducers process the actions and update the state, and then the UI components update based on the new state.

How does it work?

Synchronous Flow


Synchronous Flow

1. **Store**: Manages the application's global state.
2. **Actions**: Represents user interactions or external events.
3. **Dispatcher**: Sends actions to the reducers so they can update the store.
4. Store provides state, which is sent to the view.
5. The View triggers an Action.
6. Actions are received by the dispatcher, which forwards them to the appropriate reducer. The reducer then determines how the state should change based on the action type.

Asynchronous Flow


Async Flow

When consuming an API using Redux, the flow remains similar, but middlewares are introduced to handle asynchronous actions before they reach the reducer.

1. The view triggers an API request by dispatching an async action.
2. Middleware intercepts the action, makes the actual API call, and dispatches new actions based on the API response.
3. The appropriate reducer processes the action and updates the state.
4. Store updates and notifies view.

Redux Middleware

Middleware is code that runs between the dispatching of an action and the moment it reaches the reducer. It can add functionality like logging, API calls, or side effects.

Redux Thunk

Thunk is middleware that allows action creators to return functions instead of plain objects, useful for async logic.

For step-by-step guides with code:

- Redux Traditional Steps — JavaScript
- Redux Traditional TypeScript — TypeScript

Redux Toolkit (RTK)

Redux Toolkit is the official, recommended way to write Redux logic. It reduces boilerplate with helpers like `configureStore()`, `createSlice()`, and `createAsyncThunk()`.

See Redux Toolkit for the full reference.

Redux DevTools

Browser extension for debugging Redux state changes, actions, and time-travel debugging.

- Chrome: Redux DevTools
- Firefox: Redux DevTools